

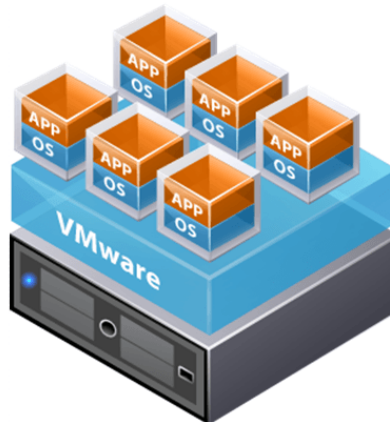
A Transmuting Journey:

From a Ruby on Rails Monolith to Elixir and Elm Microservices

Volker Rabe - Technical Lead Architect - MyMeds&Me

Reportum





The Mission



**Scale the business without
linear scaling the team**



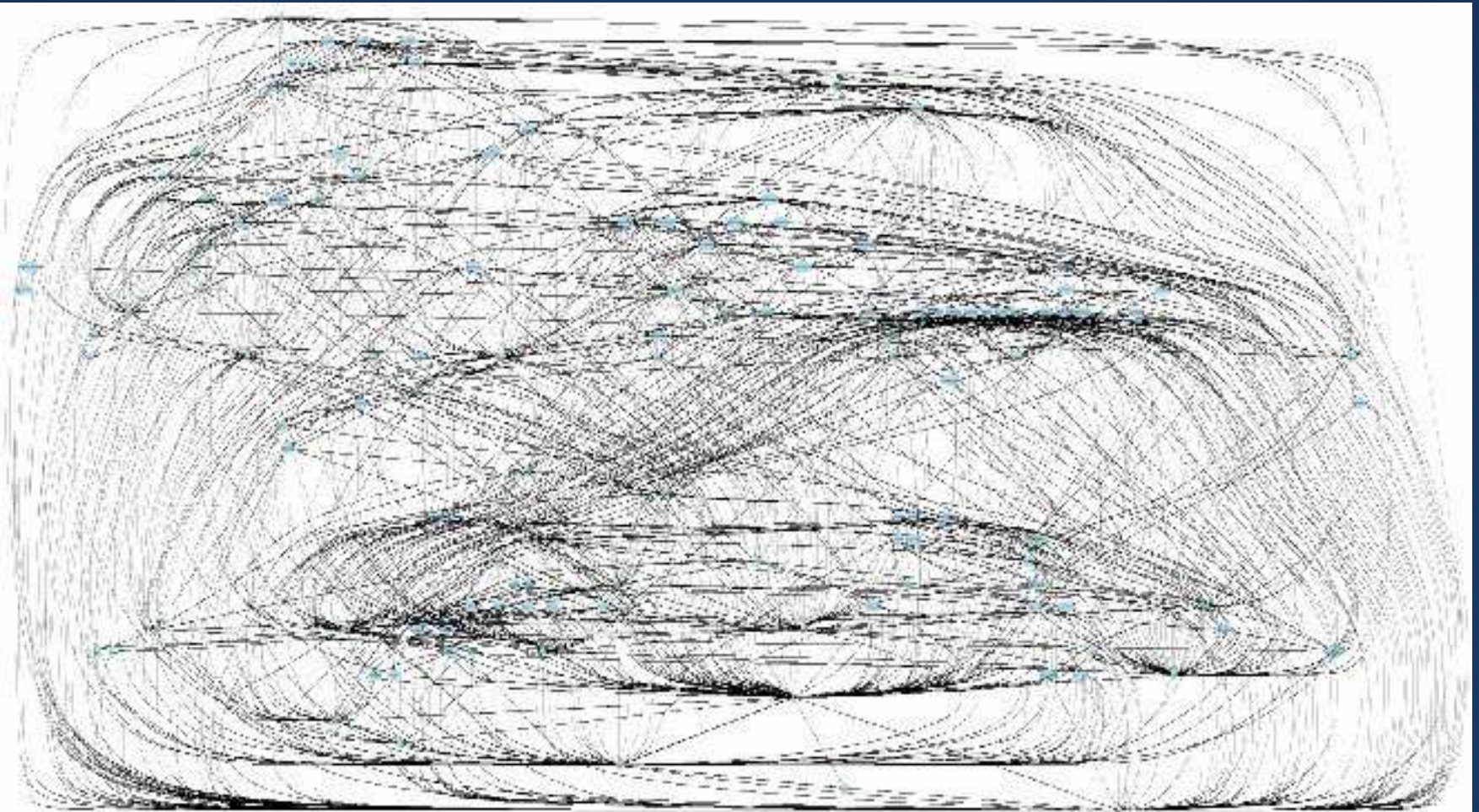
**What's wrong with
Monoliths?**



Adding more and more
functionality to the system
leads to entropy



**A new level of quantity leads
to a new level of quality**



**Entropy leads to complexity in
the code**

It get's harder and
harder to add new
functionality and
maintain the system

The Case

Growing platform with
diverging functionality per
product, version and
customer

New Quantity

Instead of 1 product, 2
customers and 2 versions
up to 4 products, 20
customers and 20 versions

New Quality

Up to 80 different versions
of the same code base
hosted on up to 320+x
hosts

Result

Platform complexity grows
exponential per product,
version and customer

Consequence

Decreasing Productivity,
Extensibility and
Maintainability

Outcome

Imploding Time to Market
and Stability plus
Exploding Costs

How should we deal
with it?

Turtle tactic



...or Raven tactic

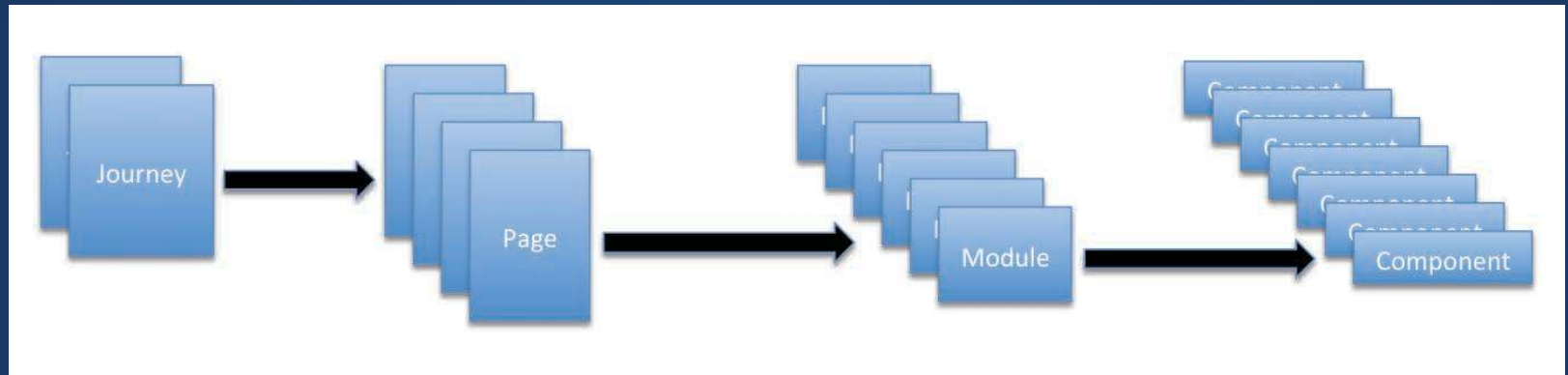
What are the
challenges we have
to face in order to
scale the business?

Configuring Tailored Products for Customers in SaaS



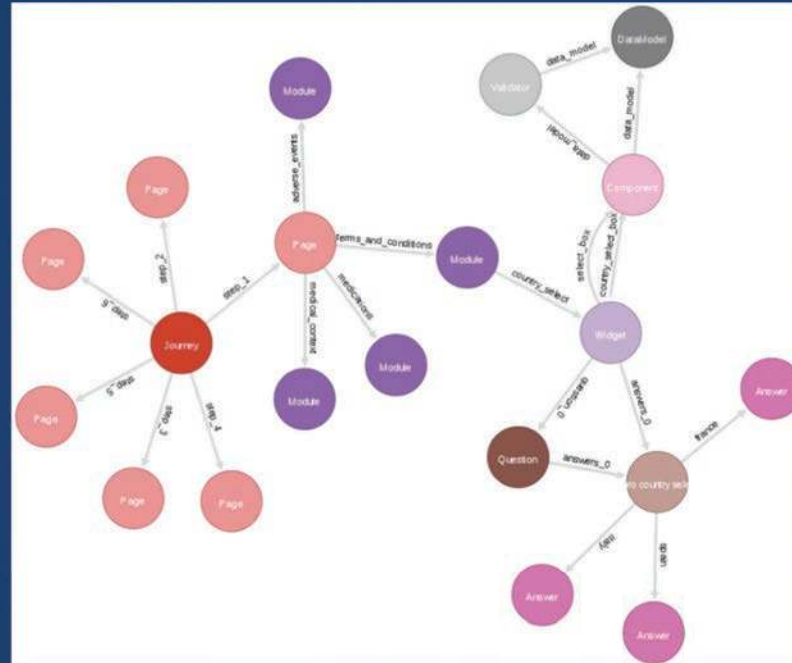
- Specific questionnaires and reports per customer
- Map questionnaires to report structure
- Increase customers without linear growing the team

Problem: Hard-coded Configuration of Questionnaires



- > Manual effort for testing changes is huge
- > Setup new questionnaires takes long
- > Maintaining the existing ones doesn't scale

Solution: Configure Tailored Products in a Graph



- > Changes can be tested in automation
- > Setup new questionnaire takes hours not days
- > Existing ones can be maintained via graph analytics

Storing Sparse Data of Customer Tailored Products

```
{
  "firstName": "John",
  "lastName": "Smith",
  "isAlive": true,
  "age": 25,
  "height_cm": 167.64,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": "10021-3100"
  },
  "phoneNumbers": [
    { "type": "home", "number": "212 555-1234" },
    { "type": "fax", "number": "646 555-4567" }
  ]
}
```

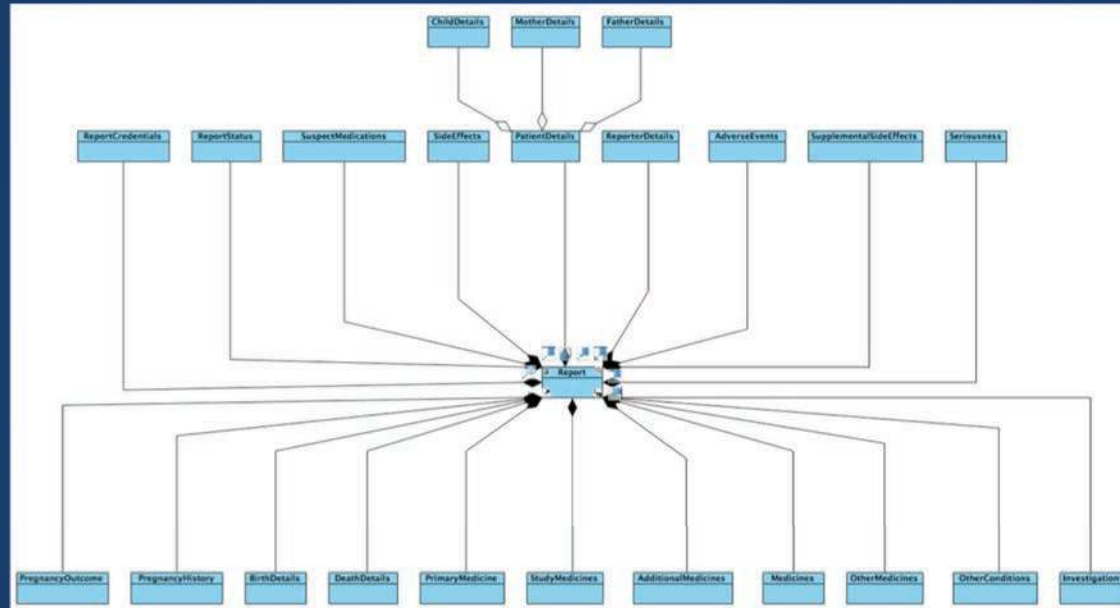
- Data structure can vary from product to product
- Data structure can vary from customer to customer
- Map form data to specific report structure

Problem: Ad hoc Format of Sparse Data

```
{ "terms_and_conditions" => { "country" => "GB", "remote_ip" => "213.218.204.34", "terms_country" => "GB", "medical_context" =>
{ "medical_context" => "This is the contact centre regression test script SIT_079 (compiled by SL. ", "product_complaint_toggle" =>
{ "value" => "no", "medical_information_toggle" => { "value" => "no" }, "reporter_details" => { "forename" => "Testerfirstnamesl", "surname" =>
"TesterSurnamesl", "country" => "GB", "contact_consent" => { "value" => "y" }, "relationship_to_patient" => "medical_doctor", "telephone" =>
{ "country_calling_code" => "44", "telephone_number" => "0170777888", "telephone_extension" => "222", "address_line_1" => "Noel Street",
"town" => "London", "county" => "United Kingdom", "postcode" => "AB1 0D2", "email" => "testerxx@mymedsandme.com", "company_aware_date" =>
{ "date" => { "date" => "22 Aug 2014" }, "medications" => { "primary" => { "toggle" => "1", "term" => { "name" => "Nymedocol", "other_indications" =>
{ "term" => { "name" => "Headache", "meddra_version" => "17.1", "l1t_code" => "1001211", "company_product" => { "value" => "n" }, "dates" =>
{ "from" => { "date" => "1 Nov 2012", "to" => { "date" => "1 Dec 2012" }, "batch" => { "number" => "123456789xyz", "expiry_date" => { "date" => "Dec
2012" }, "text_dose" => "12 mg 10 times a day", "dose_changed" => { "value" => "decreased", "dose_change_improved" => { "value" => "y",
"dose_change_reverted" => { "value" => "y" }, "dose_reversion_reoccurrence" => { "value" => "y" }, "indications" => { "value" => "other" },
"causality" => { "Headache" => "unknown", "Whole Body - Skin infection" => "no", "Spots blocking my vision" => "yes", "Rash" => "unknown" },
"additional" => { "7909612604" => { "term" => { "name" => "Avastin", "company_product" => { "value" => "n" }, "manufacturer_name" => "Roche",
"dates" => { "from" => { "date" => "Nov 2012", "to" => { "date" => "Dec 2012" }, "batch" => { "number" => "123456789xyz", "expiry_date" => { "date" =>
"Dec 2015" }, "text_dose" => "0.10 micros once every 2 days", "dose_changed" => { "value" => "no change", "indications" => { "value" =>
"Glioblastoma", "meddra_l1t_code" => "10018336", "causality" => { "Headache" => "yes", "Bone pain" => "unknown", "Scalp - Feeling hot" =>
"no", "Rash" => "yes" }, "7909616390" => { "term" => { "name" => "Paracetamol", "company_product" => { "value" => "n" }, "dates" => { "from" => { "date" =>
">2012", "to" => { "date" => "2012" }, "batch" => { "number" => "123456789xyz", "expiry_date" => { "date" => "Dec 2015" }, "text_dose" =>
"Causality", "dose_changed" => { "value" => "no change", "causality" => { "Pain gum" => "unknown", "Bone pain" => "yes" },
"has_other_suspect_medications" => "1", "adverse_events" => { "event" => { "7909578484" => { "term" => { "name" => "Headache", "meddra_version" =>
"17.1", "l1t_code" => "10019211", "dates" => { "from" => { "date" => "Unknown", "to" => { "date" => "Unknown" }, "seriousness" => { "value" =>
"medically-significant", "outcome" => { "value" => "recovered-completely", "7909654697" => { "term" => { "name" => "Whole Body - Skin
infection", "meddra_version" => "17.1", "l1t_code" => "10040072", "source" => "mm", "dates" => { "from" => { "date" => "Mar 2013", "to" =>
{ "date" => "Mar 2013" }, "seriousness" => { "value" => "disabling", "outcome" => { "value" => "persisting", "7909682530" => { "term" => { "name" =>
"Spots blocking my vision", "meddra_version" => "17.1", "l1t_code" => "10041741", "dates" => { "from" => { "date" => "Apr 2013", "to" =>
{ "date" => "Ongoing" }, "7909685919" => { "term" => { "name" => "Pain gum", "meddra_version" => "17.1", "l1t_code" => "10033404", "dates" =>
{ "from" => { "date" => "2013", "to" => { "date" => "2013" }, "seriousness" => { "value" => "not-serious", "outcome" => { "value" => "unknown",
"7909691510" => { "term" => { "name" => "Bone pain", "meddra_version" => "17.1", "l1t_code" => "10060602", "7909693078" => { "term" => { "name" =>
"Scalp - Feeling hot", "meddra_version" => "17.1", "l1t_code" => "10059800", "source" => "mm", "dates" => { "from" => { "date" => "2013", "to" =>
">{ "date" => "Ongoing" }, "seriousness" => { "value" => "death", "79095526230" => { "term" => { "name" => "Rash", "meddra_version" => "17.1",
"l1t_code" => "10037844", "dates" => { "from" => { "date" => "Feb 2013", "to" => { "date" => "Ongoing" }, "seriousness" => { "value" =>
"hospitalisation", "hospitalisation_dates" => { "from" => { "date" => "2 Mar 2013", "to" => { "date" => "2 Mar 2013", "outcome" => { "value" =>
"recovered-lasting", "enable" => "true", "patient_details_other" => { "age_picker" => { "dob_provided" => "0", "age_unit" => "y", "age" =>
"25", "country_of_incidence" => { "value" => "GB", "name" => "TesterXX", "sex" => { "value" => "m", "patient_seen_gp" => { "contact_consent" =>
{ "value" => "y", "country" => { "value" => "GB", "name" => "AndrewMM", "relationship_to_patient" => "nurse", "address_line_1" => "Noel
Street", "town" => "London", "county" => "England", "postcode" => "AB1 0D2", "telephone" => { "country_calling_code" => "44",
"telephone_number" => "0170777888", "telephone_extension" => "222", "email" => "andrew@mymedsandme.com", "death_details" =>
{ "cause_of_death" => { "term" => { "name" => "Heart attack", "meddra_version" => "17.1", "l1t_code" => "10019250", "date_of_death" => { "date" =>
"1 Jun 2013", "concomitant_medication" => { "drugs" => { "79095682698" => { "term" => { "name" => "Ibuprofen", "7909564839" => { "term" => { "name" =>
"Avanz" }, "other_conditions" => { "conditions" => { "treatment-for-allergies" => { "selected" => "1", "name" => "Treatment for allergies",
"meddra_l1t_code" => "10044059", "event" => { "79096980437" => { "term" => { "name" => "Head cold", "meddra_version" => "17.1", "l1t_code" =>
"10019192", "79095769976" => { "term" => { "name" => "Leg pain", "meddra_version" => "17.1", "l1t_code" => "10024130", "date_occurred" =>
{ "from" => { "date" => "Jan 2013", "to" => { "date" => "Ongoing" } }, "investigations" => { "investigation" => { "79095719690" => { "type" => { "name" =>
"Alanine aminotransferase (ALT/SGPT)", "meddra_l1t_code" => "10001546", "range" => { "result" => { "79095728545" => { "date" => { "date" => "Feb
2013", "value" => "28.0", "unit" => "U/L", "normal" => { "low" => "7.0", "high" => "56.0", "unit" => "U/L", "79095743751" => { "type" => { "name" =>
">Scar", "meddra_l1t_code" => "10061409", "unknown" => { "choose" => { "value" => "other", "other" => { "result" => { "79095744820" => { "date" =>
{ "date" => "Aug 2014", "result" => "investigations" }, "7909693400" => { "type" => { "name" => "Echovirus test", "meddra_l1t_code" =>
"10050608", "unknown" => { "pass_fail" => { "result" => { "79096940084" => { "date" => { "date" => "1 Jun 2013", "result" => { "value" =>
"Negative" }, "chooser" => { "value" => "pass_fail" }, "autopsy" => {}, "supplemental_investigations" => {}, "serum_sample" => {} }
```

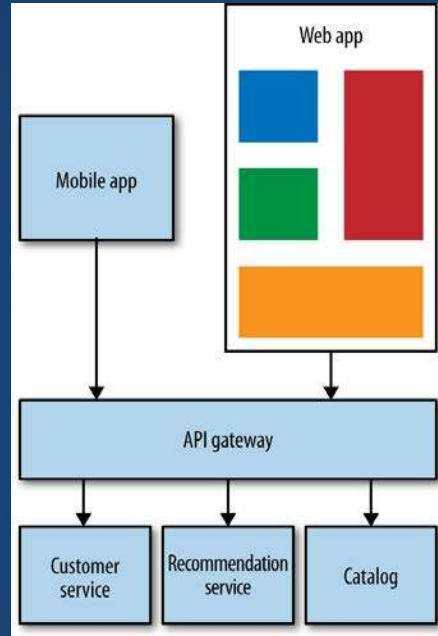
- > Changes on the data structure are not transparent
- > Diverging data structures are hard to provide
- > Querying data for analysis is non trivial

Solution: Store Sparse Data as Connected Documents in Graph



- > Changes on the data structure are controlled and transparent
- > Complex queries can be done with ease in a graph
- > Realtime analytics can run on mass data

Support Multiple Frontends



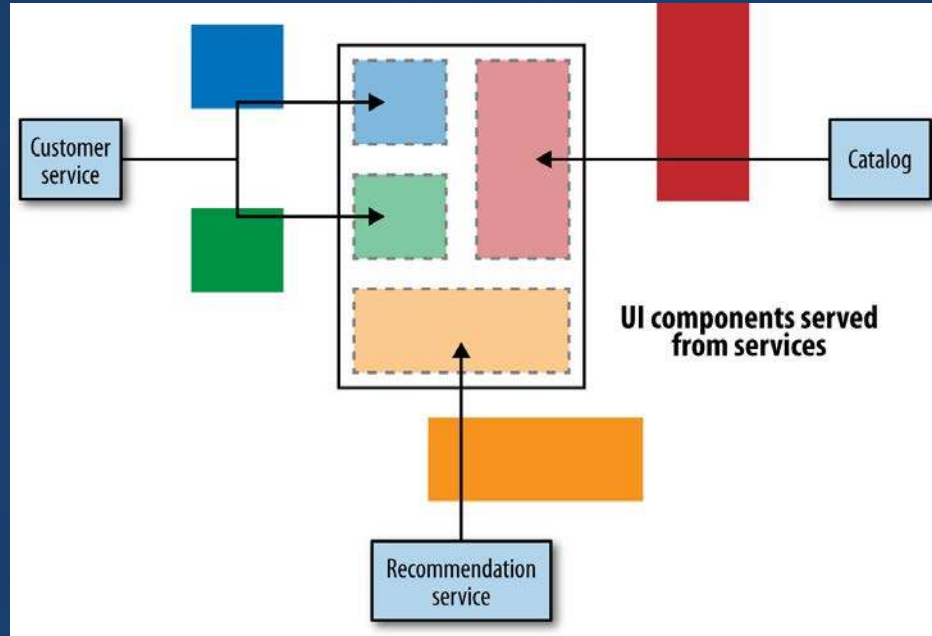
- Backend APIs for web and mobile frontends
- Same scope of functionality for all frontends
- Switch web technology without backend changes

Problem: Tight Coupling of Frontend and Backend



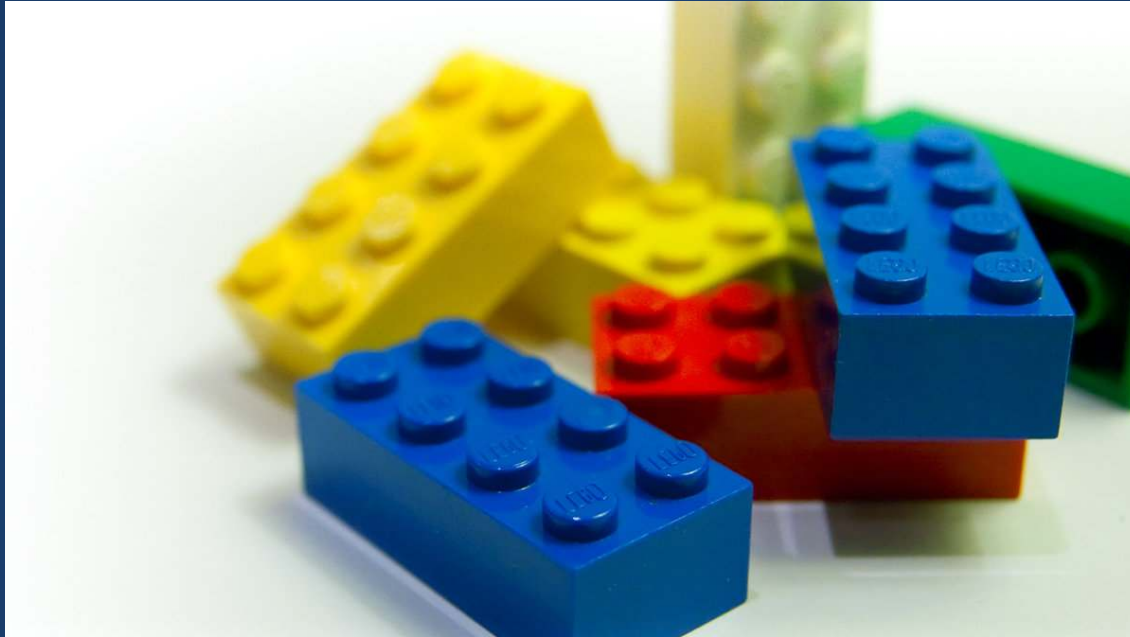
- > Changes on both ends are fragile
- > Extending functionality is pretty costly
- > Providing multiple frontends is very hard

Solution: Decouple Frontend and Backend via WebSockets



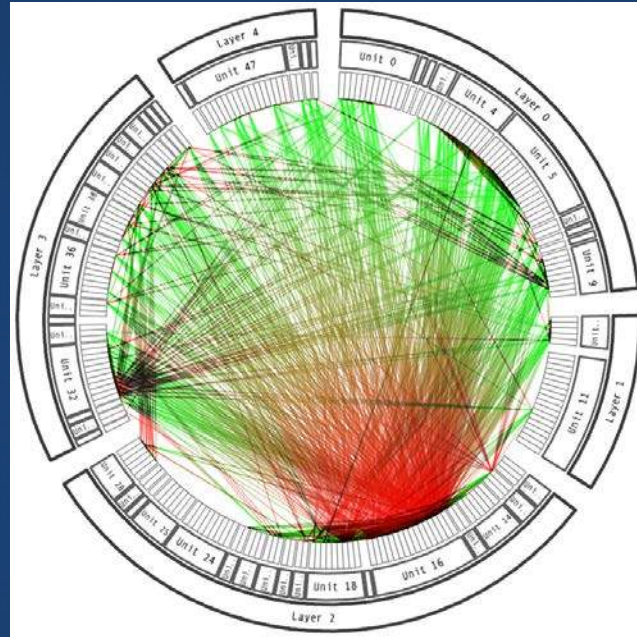
- > Frontend/Backend can be developed and tested separately
- > Extending functionality of API and UI is straight forward
- > UI can be generated for multiple frontends

Sharing Modules for Multiple Products



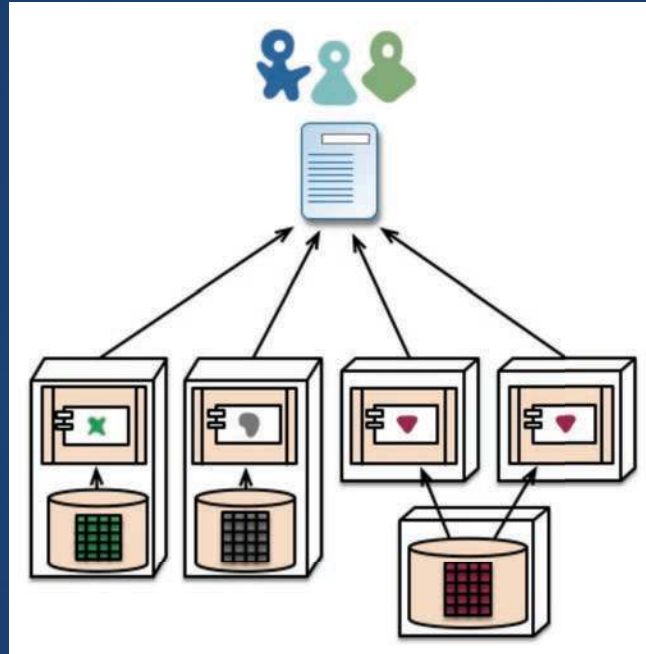
- Products have the same modules in common
- Modules have various dependencies with each other
- Modules evolve with different speed per product

Problem: Tight Coupling of Modules



- > Diverging code between products and versions
- > Refactorings have side effects on other modules
- > Need to deploy, test and ship the entire product

Solution: Decompose Application as Decoupled Microservices



- > Microservices can be developed and replaced individually
- > Microservices can be reused for products and legacy systems
- > Microservices can be tested, deployed and scaled individually

Agile Operational Approach



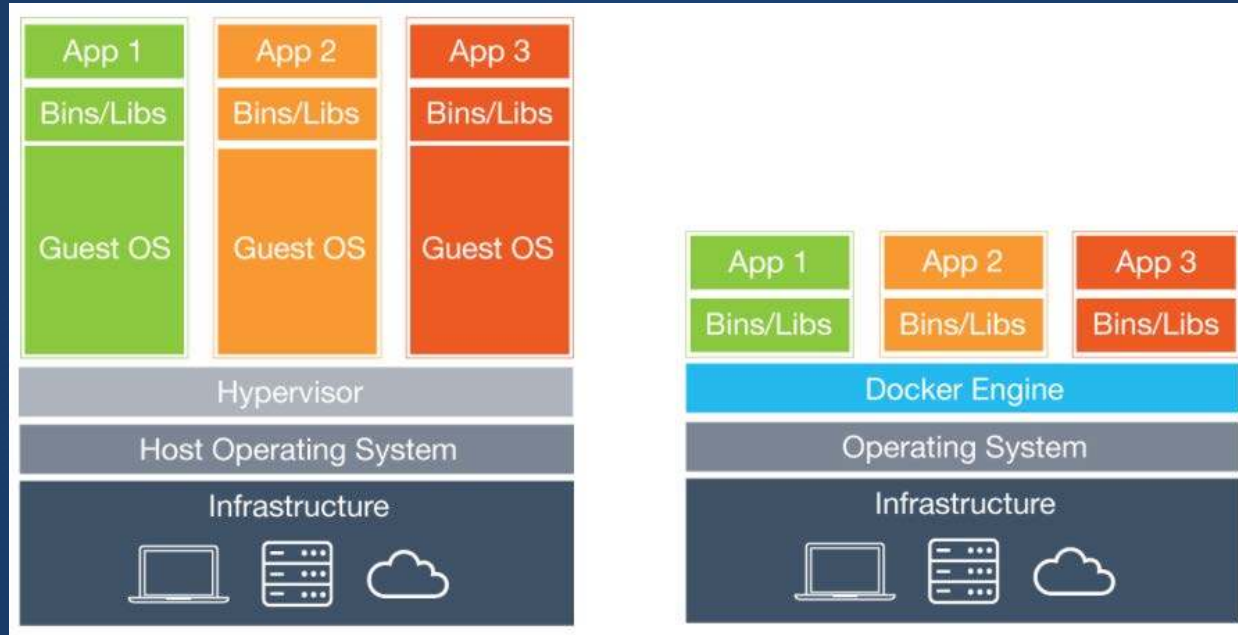
- Increase agility with DevOp culture
- Minimise the operational efforts
- Traceability over state and changes

Problem: Entropy of Mutable Infrastructure



- > Accumulated changes feed system entropy
- > Changes are hard, error prone and a time sink
- > Operations become bottleneck

Solution: Containerise Application and Infrastructure with Docker



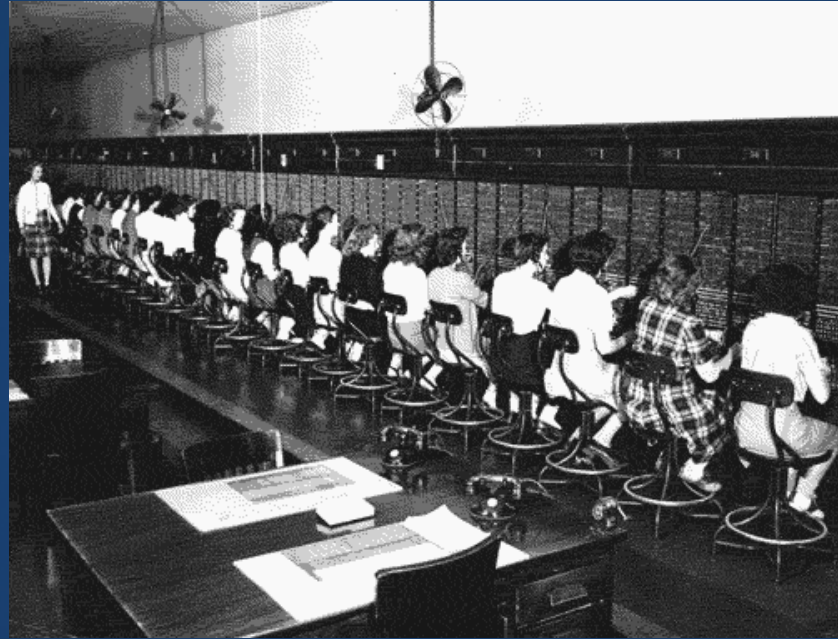
- > System provides traceability of processes, state and changes
- > Devs can manage system setup via containers
- > Operations can become a shared responsibility

Support for Multi-Customer, -Product, -Version, -Environment



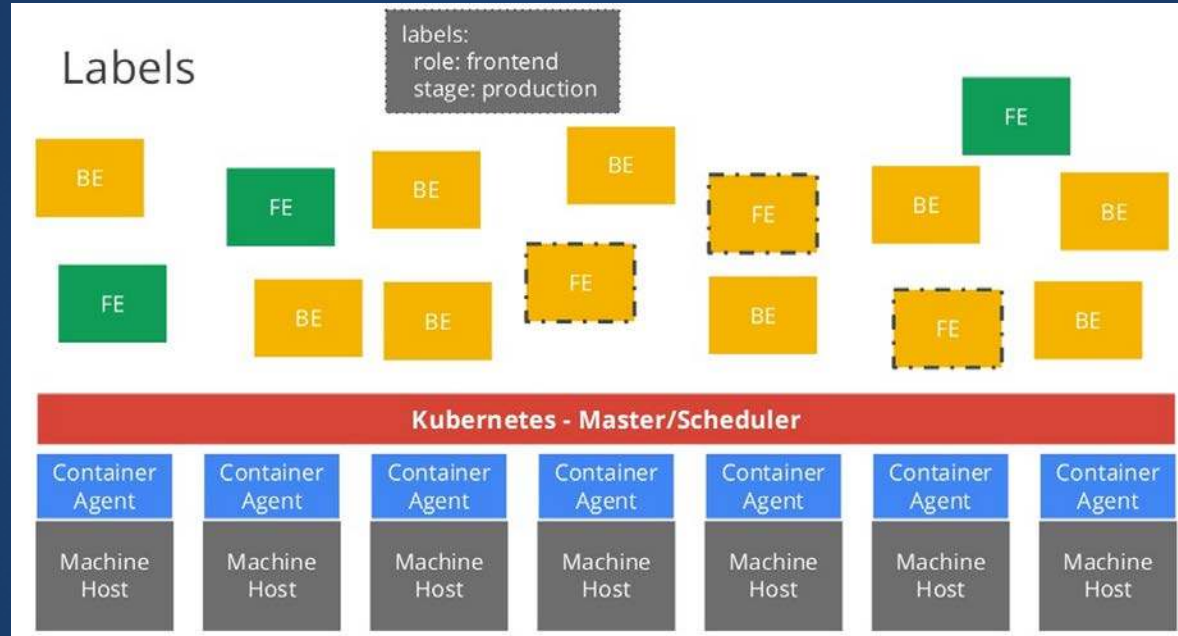
- Single-tenant hosting for customers
- One or more products per customer
- Support 2 major versions per customer

Problem: Managing Complexity



- > Up to 20 clients with up to 4 products
- > result in up to 80 versions in 4+x environments
- > spread over up 320+x hosts

Solution: Orchestrate the Containerised System with Kubernetes



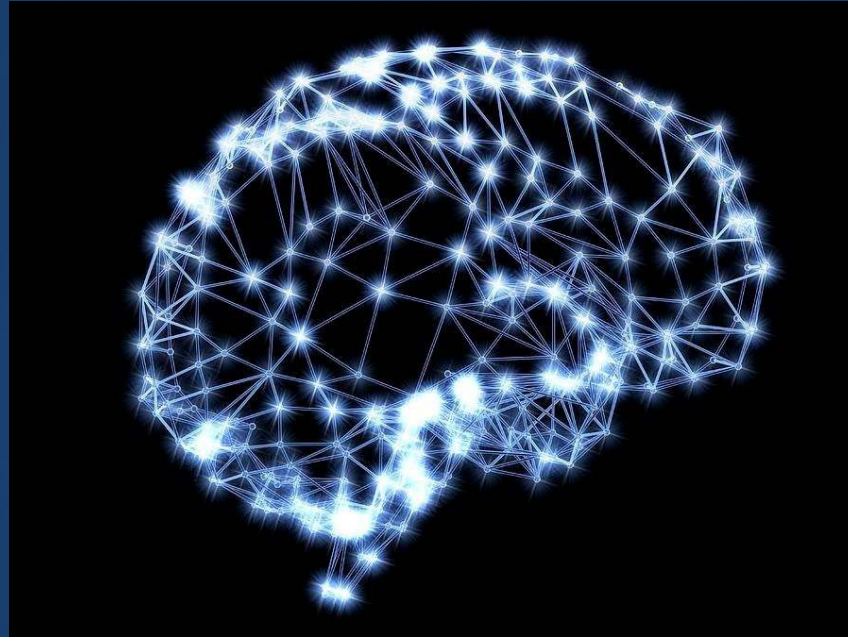
- > Versions can be handled transparently
- > Clusters can be controlled centrally for all versions
- > Releases and patches can be promoted in the clusters

Conclusion

The solutions sound
promising by themselves,
but how do they work
together?

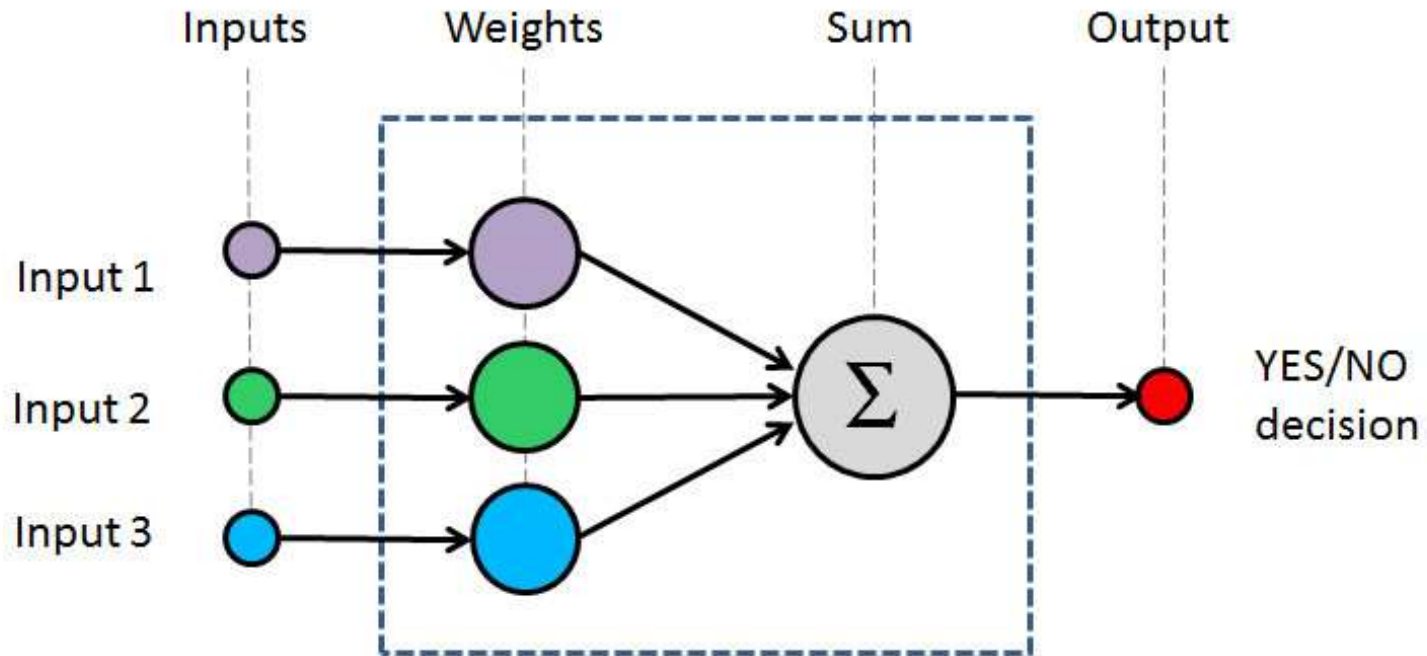
The Vision

Perceptrix

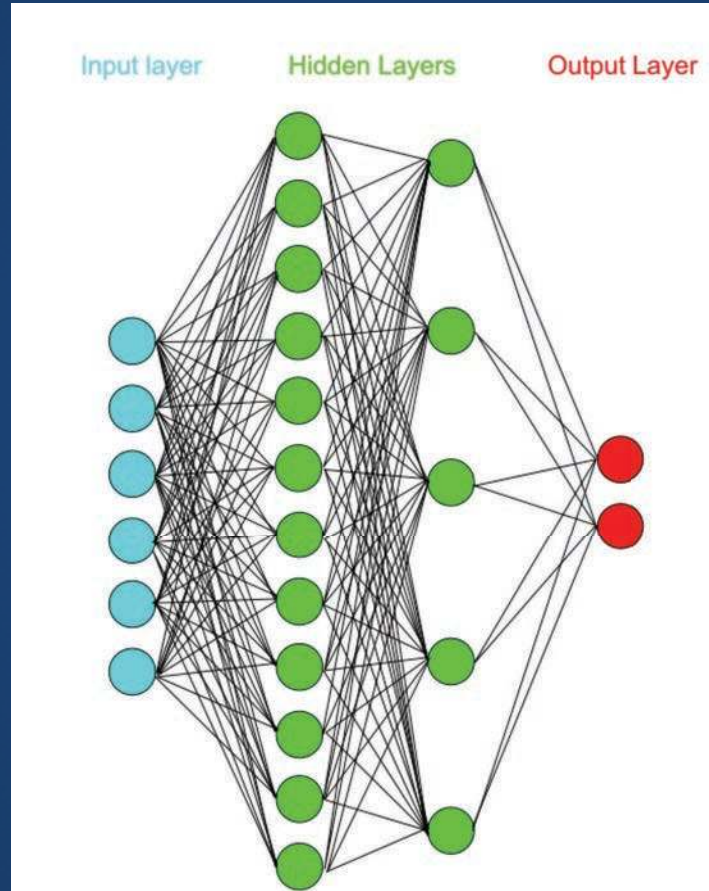


- Data Transformation
- Propagation
- Parallelisation
- Realtime

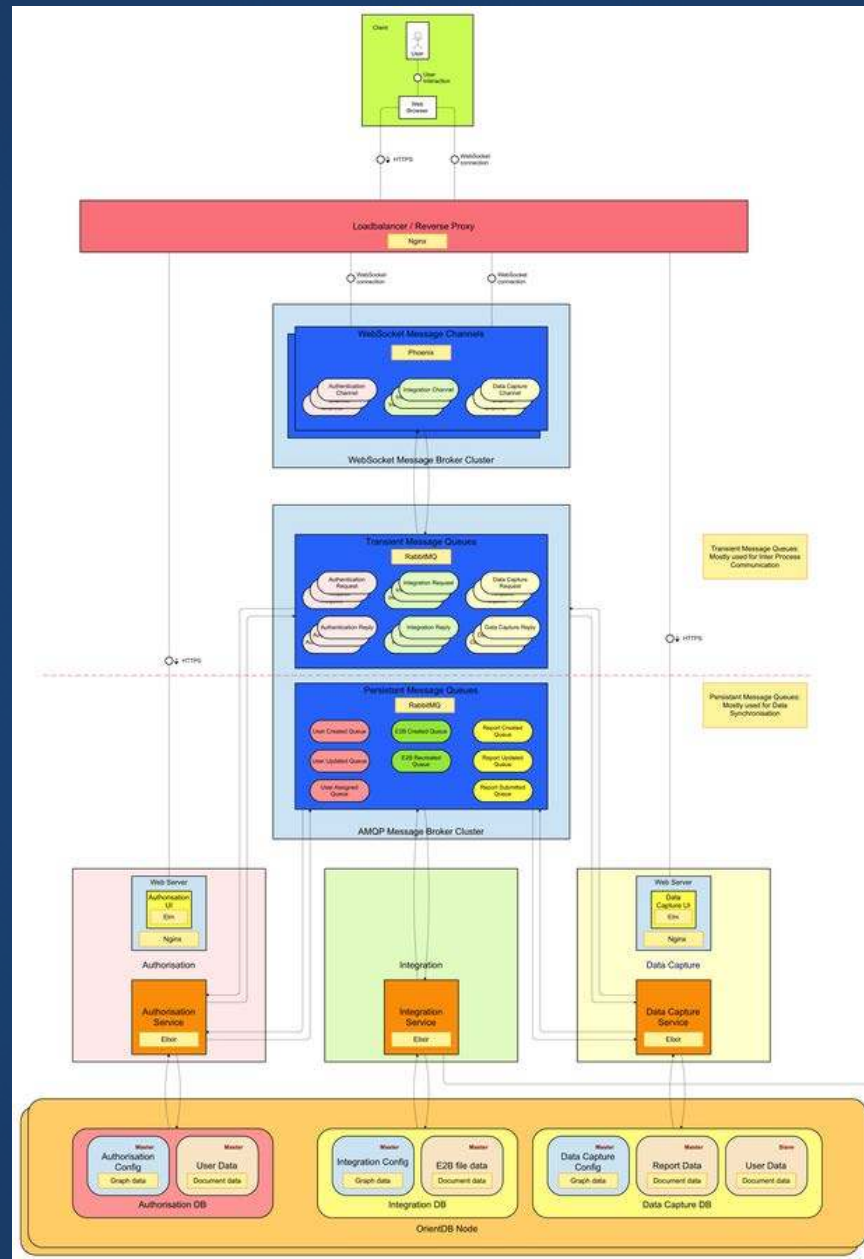
Perceptron

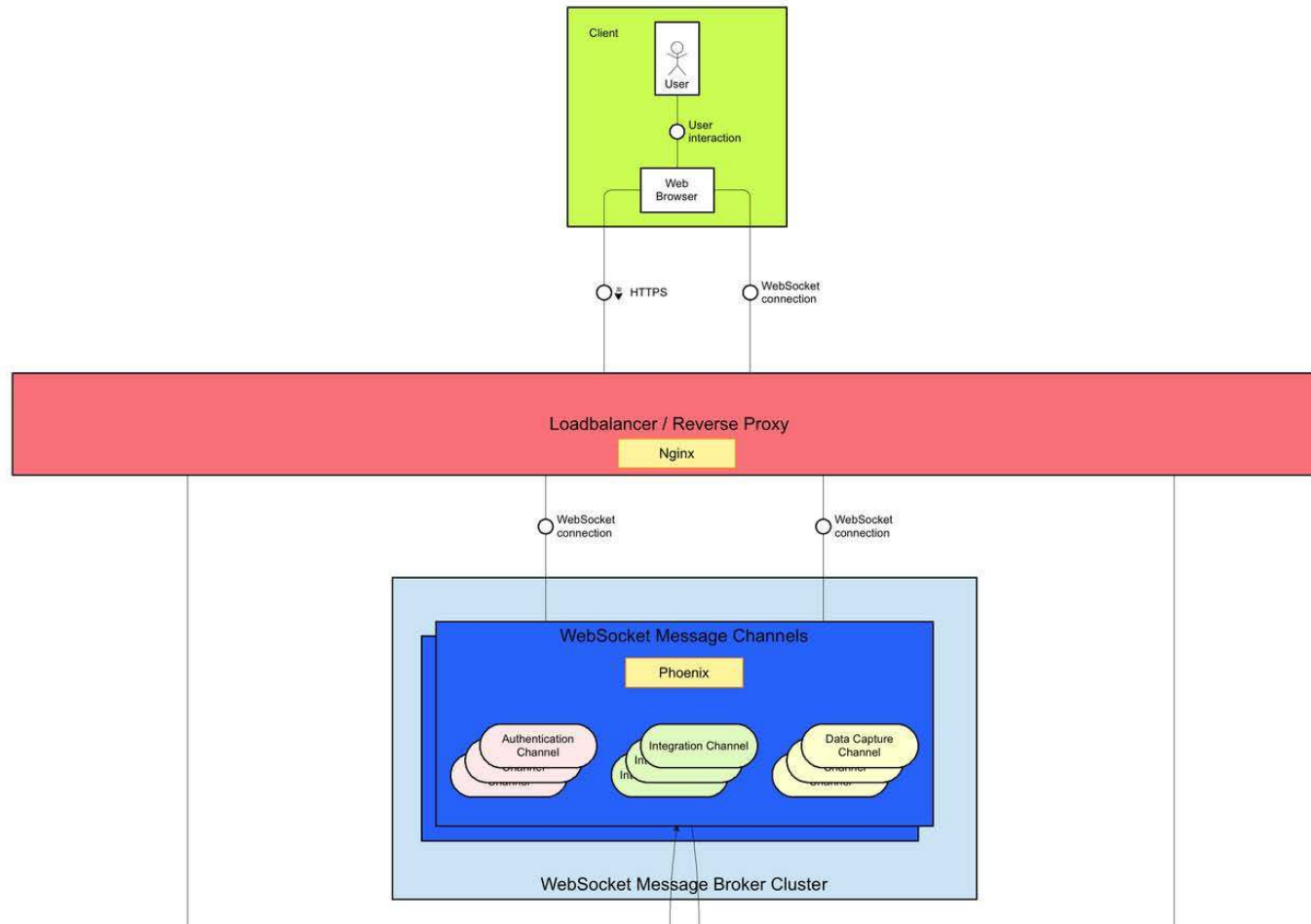


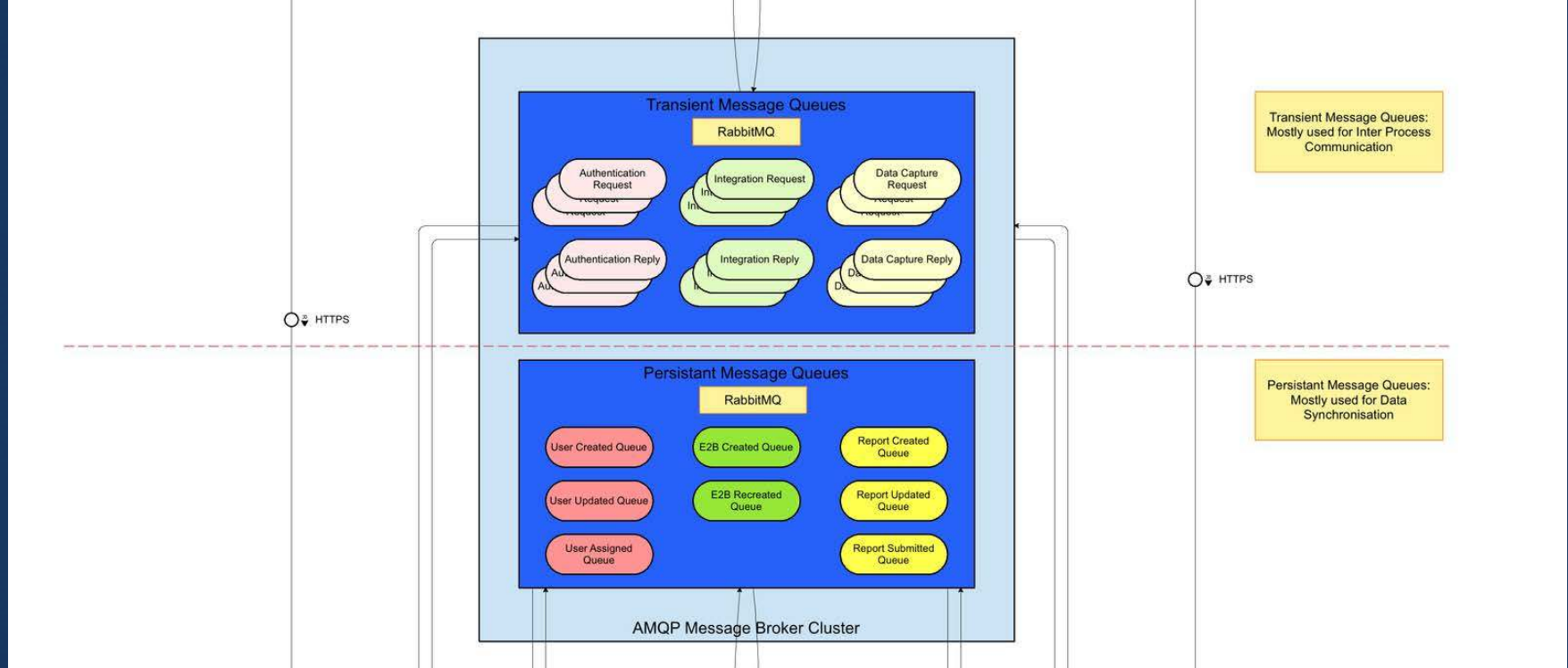
Perceptrix

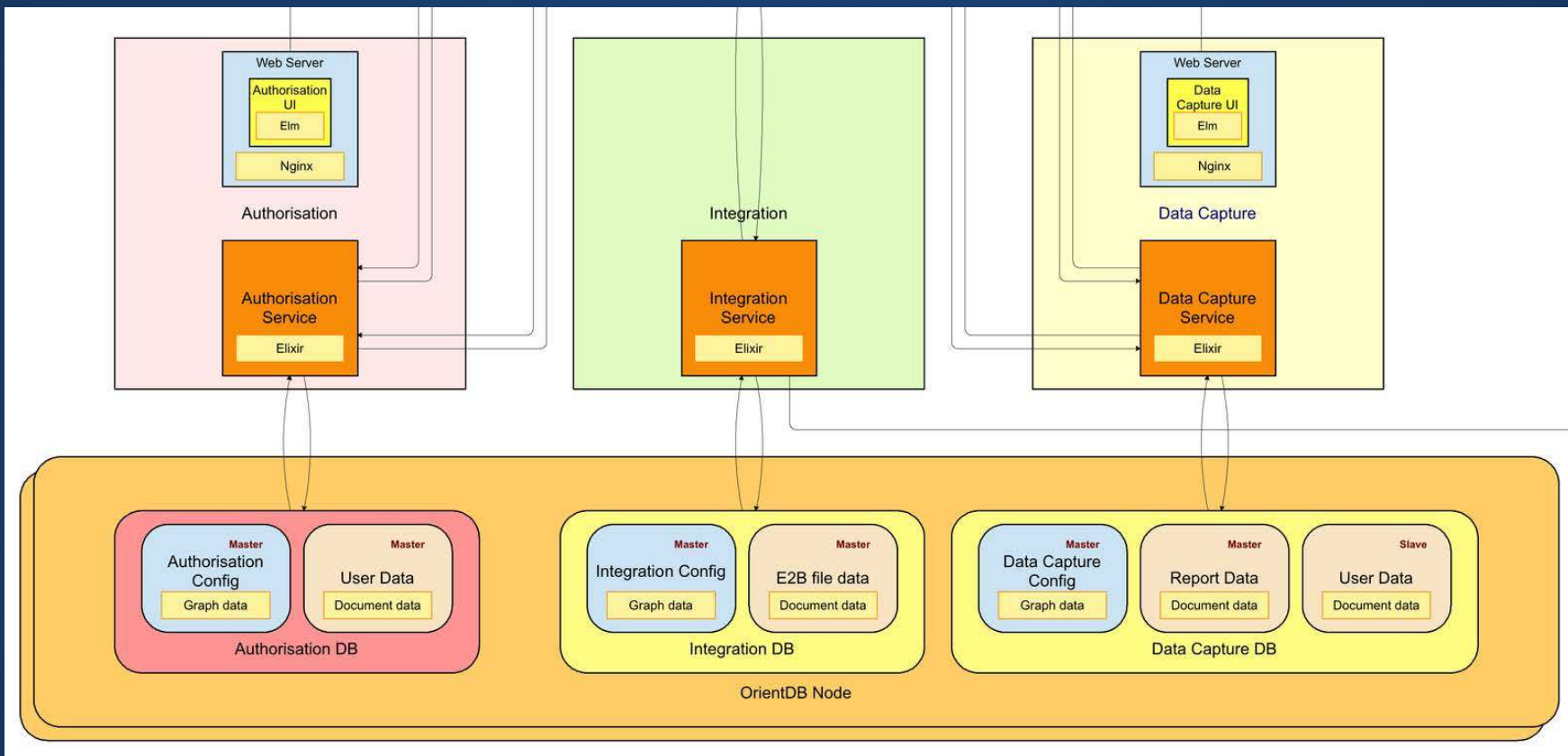


The Architecture







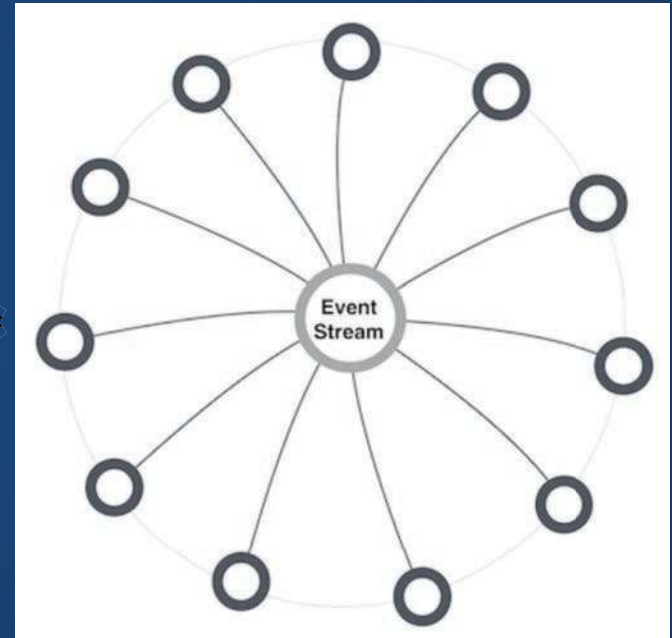
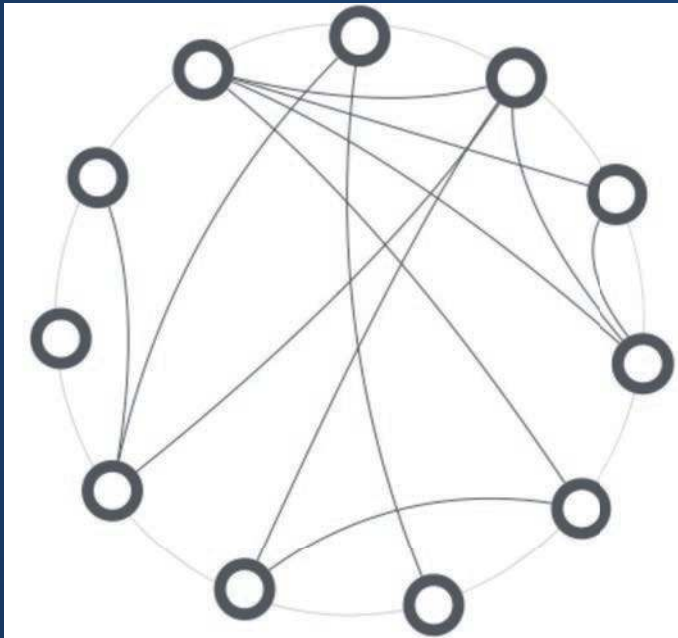


The Approach

Approach

- Why Event-Driven Architecture?
- Why Elixir Backend Services?
- Why Phoenix WebSocket Broker?
- Why RabbitMQ Message Broker?
- Why Elm Frontend Services?

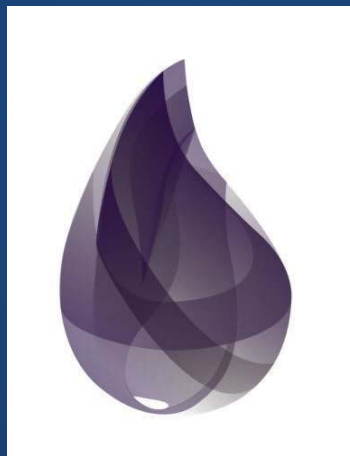
Why Event-Driven Architecture?



Why Event-Driven Architecture?

- Support synchronous, blocking RCP where necessary
 - Use asynchronous, non-blocking PubSub where possible
 - Listen to messages for multiple concerns
 - React on events in multiple microservices in parallel
 - Loose coupling between microservices
 - Facilitate changes and integration of new microservices
-
- > Supports nicely reactive realtime application
 - > Easy integration of legacy and third party systems
 - > Fits distributed architecture and Internet of Things

Why Elixir Backend Services?



Functional

```
defmodule Math do
  def square(x) do
    x * x
  end
end

Enum.map [1, 2, 3], &Math.square/1
```

Immutable State

```
iex> tuple = {:ok, "hello"}  
{:ok, "hello"}  
iex> put_elem(tuple, 1, "world")  
{:ok, "world"}  
iex> tuple  
{:ok, "hello"}
```

Pattern Matching and Recursion

```
defmodule Calculator do

  def sum(list) when is_list(list) do
    add(list)
  end

  def sum(_) do
    nil
  end

  defp add([head | tail]) do
    head + add(tail)
  end

  defp add([]) do
    0
  end
end
```

Protocols and Meta Programming

```
defprotocol Blank do
  def blank?(data)
end

defimpl Blank, for: Integer do
  def blank?(_), do: false
end

defimpl Blank, for: List do
  def blank?([]), do: true
  def blank?(_), do: false
end
```

```
defmodule TestCase do
  @doc false
  defmacro __using__(_opts) do
    quote do
      import TestCase
    end
  end

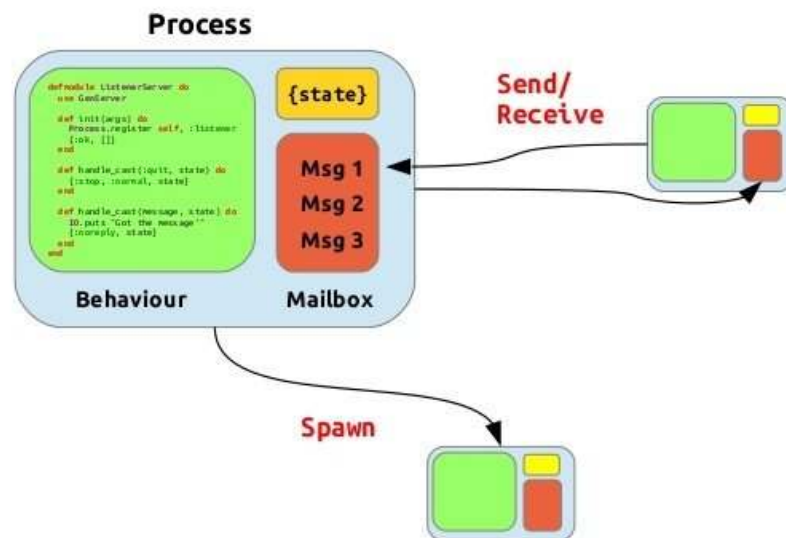
  defmacro test(description, do: block) do
    function_name = String.to_atom("test " <> description)
    quote do
      def unquote(function_name)(), do: unquote(block)
    end
  end
end
```


Concurrent Programming

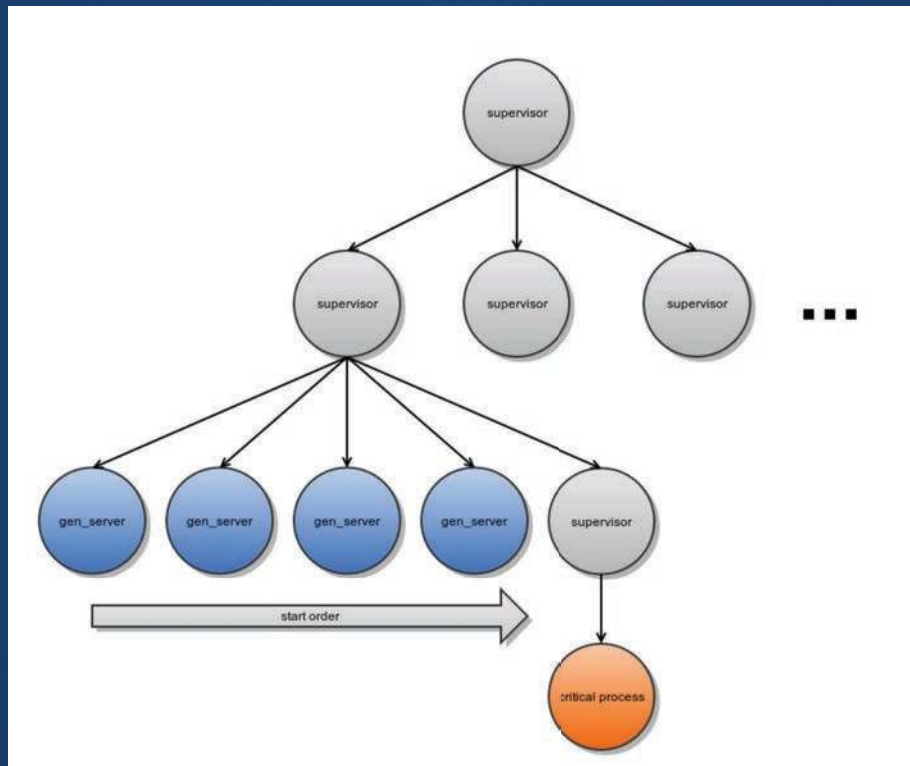


Elixir

The Actor model



Fault-tolerant

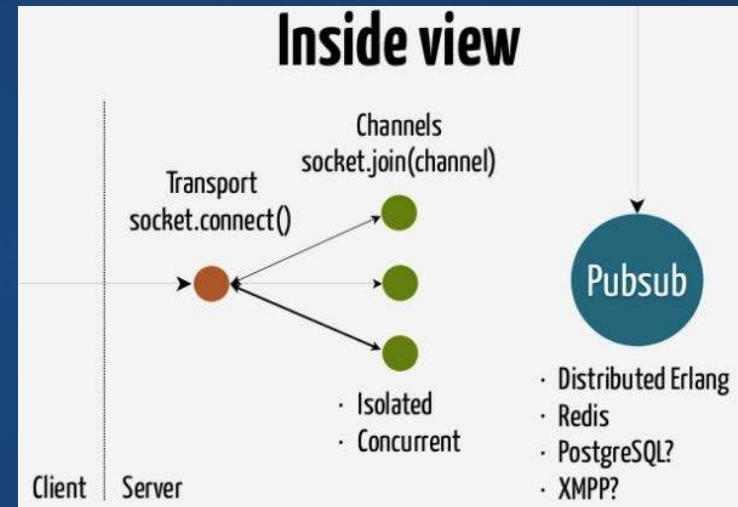
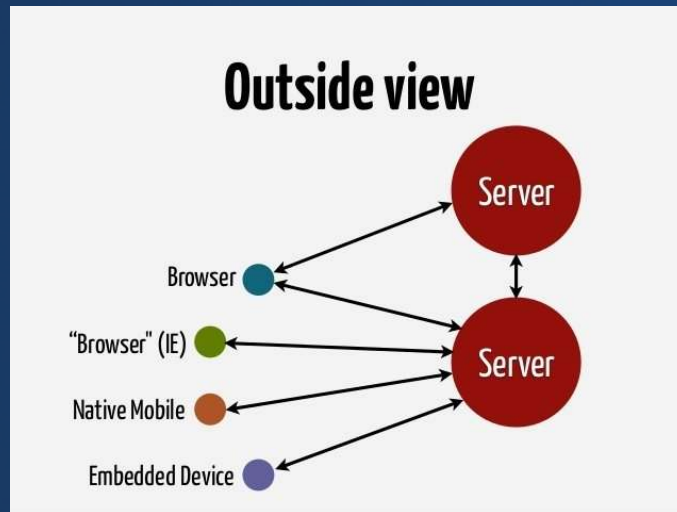


Why Elixir Backend Services?

- Functional Programming Language
- Immutable State
- Pattern Matching and Recursion
- Protocols and Meta Programming
- Concurrent Programming
- Fault-tolerant

- > High productivity and low maintenance code
- > Efficient Data Transformation made simple
- > Efficient, Scalable, Fault-tolerant Applications made simple

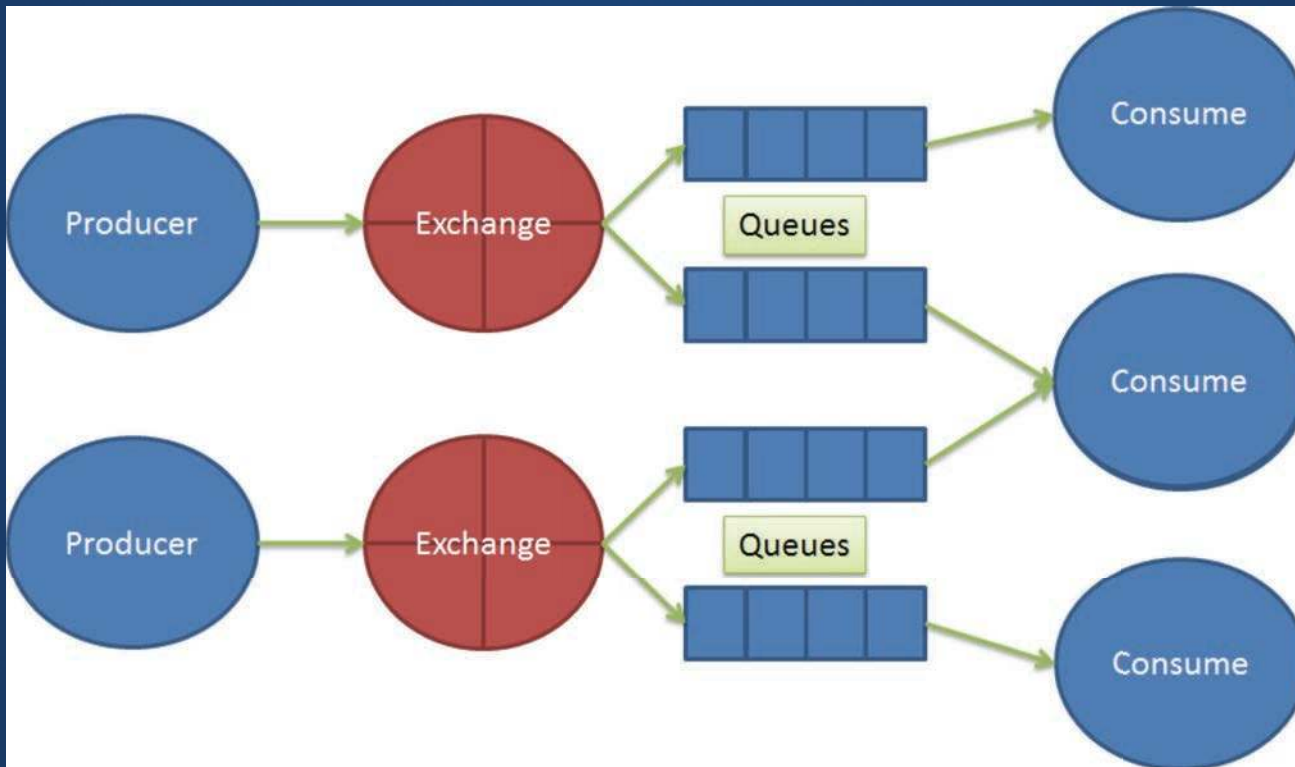
Why Phoenix WebSocket Broker?



Why Phoenix WebSocket Broker?

- Beauty of Elixir
 - Lightweight and Modular
 - Power of Plug in Controllers and Routers
 - Pattern Matching and Authorisation in Channels
 - Option to use Phoenix.PubSub with Redis
 - Scalable and Efficient WebSocket Communication
-
- > Highly available, dump and secure Broker
 - > Central instance for crosscutting non-functional concerns
 - > Scalable, performant and efficient alternative to API Gateways

Why RabbitMQ Message Broker?



Why RabbitMQ Message Broker?

- Build in Erlang with OTP
 - Real Queues: First in, First out
 - Transient and Persistent Message Queues
 - Many Patterns: RPC, PubSub, Routing, Topics, Worker Queues
 - Transactional Handling and Replays Messages
 - Management Interface
-
- > Highly available, resilient and reliable Broker
 - > Enables Inter-process calls and data synchronisation
 - > Technology independent, smart backend services

Why Elm Frontend Services?

```
import Html exposing (text)

-- Zip two lists together. In this case, we are pairing up
-- names and ages.
main =
  text (toString (zip ["Tom", "Sue", "Bob"] [45, 31, 26]))

zip : List a -> List b -> List (a,b)
zip xs ys =
  case (xs, ys) of
    ( x :: xs', y :: ys' ) ->
      (x,y) :: zip xs' ys'

    (_, _) ->
      []
```

Why Elm Frontend Services?

- Functional Programming Language
 - Static Typed and Compiler Safe
 - Stateless, Immutable Data
 - Pattern Matching, Recursion, Map, Merge, FoldP, Filter
 - Elm Architecture
 - Time Traveling Debugger
 - Semantic Versioning Package Manager
-
- > High productivity and low maintenance code
 - > Efficient data transformation made simple
 - > Event-driven architecture made simple

The Learnings

Infrastructure

- Microservices imply more complexity in infrastructure
- Complexity can be managed by isolation and automation
- Docker and Kubernetes fit isolation and automation
- Immutable infrastructure makes the difference
- New infrastructure requires a lot of ground work
- Good logging and monitoring is key
- Ecosystem is quite young and need some tweaks
- OPs need to adapt new approach to productionise it
- An agile approach can minimise the risks

Backend

- OrientDB is very powerful and leverages new possibilities
- OrientDB is still under development
- Microservices lead to good separation and autonomy
- Container automatisaton leads to streamlined setups
- Message-driven enables choreography of microservices
- Some ground work upfront necessary using Elixir
- Functional language fits in event-driven approach
- OTP makes the difference in distributed application
- Elixir was picked up with joy by every back-/frontend Dev

Frontend

- Splitting off backend/frontend leads to exchangeable clients
- Fetching dependencies with NPM is slow and not reliable
- Elm compiler, package manager, debugger make it productive
- JS libraries can still be used for interoperation
- Using JS libraries won't leverage the type safety of Elm
- Elm is still young and has breaking changes
- Functional language fits in event-driven approach
- Elm architecture makes the difference for event-driven design
- Elm was picked up with joy by every front-/backend Dev